

Guía Infor Bridge

Continuación: PARTES D a H

Desde Cloudflare Tunnel hasta integración con TechApp_Back.
Versión adaptada: laptop NO se reinicia.

Laptop	TECHSHOPCR (Windows Server)
SQL Server	MSSQLSERVER (default instance), SQL 2008 SP3
Base de datos	TechShop_App
Usuario SQL	sa — la API restringe por código
Bridge local	localhost:8787 (Node.js 18 LTS)
Dominio público	api.tsicr-api.uk (Cloudflare)
Estado al iniciar esta guía	C.1 a C.5 completados (bridge funciona en localhost)
Restricción	La laptop NO se puede reiniciar — guía adaptada para no requerirlo
Tiempo estimado	60–80 minutos (D + E + F + G sin reboot)

Documento para David Salas Villegas — TSI

Índice

0	Estado actual y reglas críticas
D	PARTE D — Cloudflare Tunnel (15-20 min)
E	PARTE E — Servicios Windows SIN reiniciar (15 min)
F	PARTE F — Verificación final desde internet (10 min)
G	PARTE G — Laptop como servidor 24/7 (15 min, sin reboot)
H	PARTE H — Integración con TechApp_Back (yo te lo armo)
A1	Apéndice A — Troubleshooting
A2	Apéndice B — Referencia rápida de endpoints y curl
A3	Apéndice C — Checklist final imprimible

0. Estado actual y reglas críticas

0.1 Lo que ya está hecho

Esta guía empieza desde donde dejaste — todo lo siguiente ya está validado en TECHSHOPCR:

TCP/IP en 1433	Test-NetConnection = True
Auth mixta + SA	Conecta desde SSMS con SQL Server Authentication
Node.js 18 LTS	Instalado, node --version = v18.x
Bridge en C:\infor-bridge	npm install ok, node server.js conecta a TechShop_App
API local responde	curl localhost:8787/health = ok:true
API lee datos reales	curl /api/tabla/X?limit=3 devuelve filas

0.2 Reglas que NO podés ignorar

[!] REGLA #1 — La laptop NO se reinicia. Esta guía está adaptada: NO te pedimos reiniciar como parte de los pasos. Lo único que NO se puede validar sin reboot es la auto-arranque al boot. Pero los servicios marcados SERVICE_AUTO_START es una garantía de Windows: arrancan al boot sin necesidad de "ensayarlo".

[X] REGLA #2 — Seguridad por código, no por SQL. El usuario sa tiene permisos totales. La API restringe en 4 capas (header API key, whitelist de tablas, filtro de palabras prohibidas, solo GET y PUT expuestos). **No modifiques esas capas para "simplificar".**

[X] REGLA #3 — NO abras puertos al firewall ni al router. El bridge solo escucha en 127.0.0.1. Cloudflare Tunnel es saliente (la laptop sale, no entra). Si abrís 1433 al exterior, exponés Infor a internet directo.

0.3 Plan de las próximas fases

```
PARTE D -> Probar el túnel A MANO (PowerShell abierto = funciona)
|
v
PARTE E -> Convertir todo en servicios Windows (sin reboot)
|
v
PARTE F -> Verificación final desde internet
|
v
PARTE G -> Laptop 24/7 (power, lid, Windows Update, UPS)
|
v
PARTE H -> Integración con TechApp_Back (yo te lo armo)
```

D. PARTE D — Cloudflare Tunnel

Tiempo: **15-20 minutos**. Resultado: en tu celular con datos móviles, `https://api.tsicr-api.uk/health` devuelve `{"ok":true,...}` con datos reales.

D.0 Pre-condiciones

Antes de empezar PARTE D necesitás:

Bridge funcionó en C.5	Pudiste hacer <code>curl /api/tabla/...</code> y devolvió filas
node server.js está DETENIDO	Cerraste con <code>Ctrl+C</code> la ventana donde corría
Logueado como administrador	PowerShell muestra <code>PS C:\Users\administrator></code>
tsicr-api.uk en estado Active	<code>dash.cloudflare.com</code> muestra el dominio con punto verde
Internet funciona	Podés abrir <code>google.com</code> en el navegador

D.2 Instalar cloudflared

PASO D.2.1 Bajar el instalador MSI

En el navegador de la laptop, andá a:

```
https://github.com/cloudflare/cloudflared/releases/latest
```

En la sección **Assets**, bajá:

```
cloudflared-windows-amd64.msi
```

[!] NO bajes el .exe (es portable, no se registra en el PATH). NO bajes la versión 386 (es 32-bit; vos tenés 64-bit).

PASO D.2.2 Ejecutar el instalador

Doble click sobre el .msi. Aparece un asistente:

- **Next » Install** (UAC pide permisos » **Yes**) » **Finish**.
- **CIERRA TODAS las ventanas de PowerShell que tengas abiertas**. Sin esto el PATH no se actualiza en sesiones existentes.
- Abrí una ventana NUEVA de PowerShell.

PASO D.2.3 Verificar la instalación

```
cloudflared --version
```

[OK] Esperás algo como:

```
cloudflared version 2024.10.0 (built 2024-10-15-1234 UTC)
```

Si dice **"is not recognized"**: cerra PowerShell y abrila otra vez. Si igual no funciona, reinicia la laptop (la única excepción a la regla del no-reboot — es para que tome el PATH; podés posponerlo si tenés alguna sesión activa de Infor).

D.3 Login + crear el túnel

PASO D.3.1 Login en Cloudflare

```
cloudflared tunnel login
```

Qué pasa:

- Se abre el navegador automáticamente con la lista de tus dominios.
- **Click en** `tsicr-api.uk`.
- Aparece pantalla de autorización » click en **Authorize**.
- La pantalla se pone verde: "You have successfully authorized cloudflared!"
- Volvé a PowerShell » tras 2-3 segundos:

[OK] La consola dice:

```
You have successfully logged in.  
If you wish to copy your credentials to a server, they have been saved to:  
C:\Users\administrator\.cloudflared\cert.pem
```

Verifica:

```
Test-Path "$env:USERPROFILE\.cloudflared\cert.pem"  
# True
```

Si el navegador NO se abre solo, copió la URL larga que aparece en consola y pegala manualmente.

PASO D.3.2 Crear el túnel

```
cloudflared tunnel create infor-bridge
```

[OK] Esperás (tu UUID será diferente):

```
Tunnel credentials written to C:\Users\administrator\.cloudflared\
7f3a5b8c-1234-5678-90ab-cdef01234567.json.
Created tunnel infor-bridge with id
7f3a5b8c-1234-5678-90ab-cdef01234567
```

[!] **ANOTA TU UUID** en un Notepad. Lo vas a usar 2 veces en D.4. Si lo perdés, podés recuperarlo con `cloudflared tunnel list`.

Verificá que existen los 2 archivos en `.cloudflared`:

```
Get-ChildItem "$env:USERPROFILE\.cloudflared\"
```

[OK] Tenés que ver:

```
Mode LastWriteTime Length Name
----
-a--- 29/4/2026 10:30 a.m. 1234 cert.pem
-a--- 29/4/2026 10:35 a.m. 345 7f3a5b8c-...json
```

D.4 Crear el config.yml

PASO D.4.1 Abrir Notepad apuntando al archivo

```
notepad "$env:USERPROFILE\.cloudflared\config.yml"
```

Notepad pregunta si crear el archivo » **Yes**.

PASO D.4.2 Pegar el contenido EXACTO

Reemplazá el UUID por el TUYO en los 2 lugares marcados:

```
tunnel: 7f3a5b8c-1234-5678-90ab-cdef01234567
credentials-file:
C:\Users\administrator\.cloudflared\7f3a5b8c-1234-5678-90ab-cdef01234567.json

ingress:
- hostname: api.tsicr-api.uk
service: http://localhost:8787
- service: http_status:404
```

[X] 3 cosas que rompen si las hacés mal:

- 1) El UUID de tunnel: y del archivo .json deben coincidir.
- 2) Indentación con ESPACIOS, no tabs (YAML rechaza tabs).
- 3) El último - service: http_status:404 NO es opcional.

Esquema visual de espacios:

```
ingress: <- sin espacios
- hostname: api.tsicr-api.uk <- 2 espacios + '- '
service: http://localhost:8787 <- 4 espacios
- service: http_status:404 <- 2 espacios + '- '
```

Ctrl+S para guardar y cerrá Notepad.

PASO D.4.3 Validar la sintaxis del config

```
cloudflared tunnel ingress validate
```

[OK] Si está bien:

```
Validating rules from C:\Users\administrator\.cloudflared\config.yml
OK
```

Si tira error, los 2 mas comunes:

- *error parsing YAML* » tabs en lugar de espacios. Reescribí los espacios a mano.
- *tunnel ID does not match* » el UUID en tunnel: no coincide con el del archivo .json.

D.5 Apuntar el DNS

PASO D.5.1 Crear la ruta DNS

```
cloudflared tunnel route dns infor-bridge api.tsicr-api.uk
```

[OK] Esperás:

```
INF Added CNAME api.tsicr-api.uk which will route to this tunnel
tunnelID=7f3a5b8c-1234-5678-90ab-cdef01234567
```

PASO D.5.2 Verificar en el dashboard

- Andá a dash.cloudflare.com » click en tsicr-api.uk.
- Menú izquierdo: **DNS » Records**.
- Tenés que ver el CNAME nuevo:

```
Type Name Content Proxy status TTL
CNAME api 7f3a5b8c-...cfargotunnel.com [proxied] Auto
```

[!] La nubecita del Proxy debe estar **NARANJA** (Proxied), no gris (DNS only). Sin esto, el HTTPS no funciona. Si está gris, click sobre la nube y cambiala.

Esperá ~30 segundos para que propague antes de seguir.

D.6 La prueba de fuego — desde el celular

Vas a tener **2 ventanas de PowerShell** corriendo en paralelo.

PASO D.6.1 Ventana 1 - levantar el bridge

Abrí una nueva ventana de PowerShell y ejecutá:

```
cd C:\infor-bridge
node server.js
```

[OK] Esperás:

```
[OK] Conectado a SQL Server 2008 -- BD: TechShop_App
[OK] Tablas READ: ...
[OK] Tablas WRITE: ...
=====
Infor bridge escuchando en 127.0.0.1:8787
=====
```

[!] NO CIERRES esta ventana. Si la cerrás, el bridge muere.

PASO D.6.2 Ventana 2 - levantar el túnel

Abri **OTRA** ventana de PowerShell (sin cerrar la primera):

```
cloudflared tunnel run infor-bridge
```

[OK] Tenés que ver 4 conexiones registradas:

```
INF Starting tunnel tunnelID=7f3a5b8c-...
INF Connection abc123 registered connIndex=0 ip=... location=mia07
INF Connection def456 registered connIndex=1 ip=... location=mia08
INF Connection ghi789 registered connIndex=2 ip=... location=mia09
INF Connection jkl012 registered connIndex=3 ip=... location=mia10
```

[OK] Cuando ves los 4 'Connection registered', el túnel está conectado a Cloudflare. NO cierres esta ventana tampoco.

Si tira error de **UDP/QUIC** (firewall corporativo bloqueando):

```
# Cancelá con Ctrl+C y volvé a probar con HTTP/2:
cloudflared tunnel run --protocol http2 infor-bridge
```

PASO D.6.3 Test sanidad desde la laptop

Abri una **tercera** ventana de PowerShell:

```
curl.exe https://api.tsicr-api.uk/health
```

[OK] Si el túnel funciona internamente:

```
{"ok":true,"db":"TechShop_App","uptime":42,"ts":"2026-04-29T..."}
```

PASO D.6.4 TEST CRÍTICO - desde el celular con datos móviles

[X] **HACELO DESDE EL CELULAR, NO DESDE LA LAPTOP.** Desactivá WiFi y usá datos móviles (4G/5G). Solo así comprobás que el tráfico va por internet de verdad.

- Desactivá WiFi del celular.
- Confirmá que tenés señal de datos móviles.
- Abri el navegador del celular.
- Andá a:

```
https://api.tsicr-api.uk/health
```

[OK] Tenés que ver:

```
{"ok":true,"db":"TechShop_App","uptime":127,"ts":"2026-04-29T..."}
```

Si lo ves: **SQL Server 2008** está accesible via **HTTPS** publica sin VPN, sin abrir puertos, sin tocar el router.

PASO D.6.5 Test con API key (lectura real de la BD)

Volvé a la tercera ventana de PowerShell en la laptop:

```
$T = "PEGA_ACA_TU_API_KEY"
curl.exe -H "x-api-key: $T" "https://api.tsicr-api.uk/api/tabla/BridgeTest?limit=3"
```

[OK] Esperás:

```
{"rows":[{"Id":1,"Descripcion":"fila uno",...},...],"count":3,...}
```

D.7 Antes de pasar a PARTE E

[i] Cuando termines D.6 vas a tener 3 ventanas de PowerShell:

Ventana 1 con node server.js — DETENELA con Ctrl+C.

Ventana 2 con cloudflared — DETENELA con Ctrl+C.

Ventana 3 con curl — podés cerrarla.

En PARTE E vas a convertir Ventana 1 y 2 en servicios Windows que sobreviven cierre de PowerShell.

Errores típicos en PARTE D y solución rápida:

Error 1033	Túnel no activo » verificar Ventana 2
Error 502 Bad Gateway	Bridge (Ventana 1) caído » verificar
Error 530	config.yml apunta a puerto/host equivocado
DNS_PROBE_FINISHED_NXDOMAIN	Esperar 2 min más por propagación
Error 1016 Origin DNS	Nube en Cloudflare está gris » ponela naranja (D.5.2)

E. PARTE E — Servicios Windows (sin reiniciar)

Tiempo: **15 minutos**. Resultado: bridge + cloudflared corriendo como servicios Windows. Podés cerrar todas las ventanas, desconectarte de RDP, hasta cerrar sesión, y la API sigue respondiendo.

[OK] **Por qué NO necesitás reiniciar:** los servicios marcados como SERVICE_AUTO_START arrancan al boot — es una **garantía de Windows**, no algo que tengas que validar manualmente. Cuando algún día puedas reiniciar la laptop (mantenimiento programado, finde, etc), todo va a arrancar solo. Los tests de E.3 demuestran que sobrevive todo lo demás.

E.0 Antes de empezar

Detené las 2 ventanas de PowerShell que dejaste corriendo en D.6:

- Ventana 1 (node server.js) » **Ctrl+C** » cerrar.
- Ventana 2 (cloudflared tunnel run) » **Ctrl+C** » cerrar.

E.1 Bridge como servicio Windows (NSSM)

PASO E.1.1 Bajar e instalar NSSM

- Andá a nssm.cc/download
- Bajá **nssm-2.24.zip**.
- Click derecho sobre el zip » **Extraer todo** » destino: C:\nssm\

Verificá:

```
Test-Path "C:\nssm\win64\nssm.exe"
# True
```

Si te dio False, revisá que la estructura quedó como C:\nssm\win64\nssm.exe. Algunos zips meten todo en nssm-2.24\; movelo un nivel arriba si hace falta.

PASO E.1.2 Crear el servicio (PowerShell admin)

[!] Abrió PowerShell **como Administrador**: click derecho en el ícono » Run as administrator.

Pegá los comandos uno por uno:

```
C:\nssm\win64\nssm.exe install InforBridge "C:\Program Files\nodejs\node.exe"
"C:\infor-bridge\server.js"
C:\nssm\win64\nssm.exe set InforBridge AppDirectory "C:\infor-bridge"
C:\nssm\win64\nssm.exe set InforBridge AppStdout "C:\infor-bridge\service-out.log"
C:\nssm\win64\nssm.exe set InforBridge AppStderr "C:\infor-bridge\service-err.log"
C:\nssm\win64\nssm.exe set InforBridge Start SERVICE_AUTO_START
C:\nssm\win64\nssm.exe set InforBridge AppStopMethodConsole 10000
C:\nssm\win64\nssm.exe set InforBridge AppExit Default Restart
```

Cada uno debe responder Set parameter "X" successfully.

PASO E.1.3 Auto-restart si crashea

```
sc.exe failure InforBridge reset= 0 actions= restart/5000/restart/5000/restart/5000
```

[OK] Esperás:

```
[SC] ChangeServiceConfig2 SUCCESS
```

Si el proceso muere por cualquier razón, Windows lo reinicia a los 5 segundos. Y si vuelve a morir, otra vez. Para siempre.

PASO E.1.4 Arrancar el servicio

```
Start-Service InforBridge  
Start-Sleep -Seconds 3  
Get-Service InforBridge
```

[OK] Esperás:

```
Status Name DisplayName  
-----  
Running InforBridge InforBridge
```

PASO E.1.5 Validar que respondió

```
curl.exe http://localhost:8787/health
```

[OK] Esperás:

```
{"ok":true,"db":"TechShop_App",...}
```

Si dice **Stopped** o **curl falla**: mira los logs:

```
Get-Content C:\infor-bridge\service-err.log -Tail 30
```

Error más común: NSSM no encontró `node.exe`. Verificá la ruta:

```
Get-Command node | Select-Object Source  
# Source : C:\Program Files\nodejs\node.exe
```

Si la ruta es distinta, recreá el servicio editando con `nssm.exe edit InforBridge`.

E.2 Cloudflared como servicio Windows

PASO E.2.1 Instalar como servicio

```
cloudflared service install
```

[OK] Esperás:

```
INF Using Systemd  
INF Service installed
```

PASO E.2.2 Arrancar y verificar

```
Start-Service cloudflared
Start-Sleep -Seconds 5
Get-Service cloudflared
```

[OK] Esperás:

```
Status Name DisplayName
-----
Running cloudflared Cloudflare Tunnel agent
```

PASO E.2.3 Auto-restart

```
sc.exe failure cloudflared reset= 0 actions= restart/5000/restart/5000/restart/5000
```

PASO E.2.4 Validar que el túnel está conectado

```
cloudflared tunnel list
```

La columna **CONNECTIONS** debe mostrar conexiones activas (típicamente 4):

[OK] Esperás:

```
ID NAME CREATED CONNECTIONS
7f3a5b8c-... infor-bridge 2026-04... 4xmia
```

[!] Si dice **0 conexiones**: el servicio no encontró el config.yml. Cuando se instala como servicio, busca el config en otra ruta. Solución:

```
$src = "$env:USERPROFILE\.cloudflared"
$dst = "C:\Windows\System32\config\systemprofile\.cloudflared"
New-Item -ItemType Directory -Path $dst -Force
Copy-Item "$src\*" $dst -Force
Restart-Service cloudflared
Start-Sleep -Seconds 5
cloudflared tunnel list
```

E.3 Validación SIN reiniciar

Estas 4 pruebas demuestran que los servicios sobreviven todos los escenarios EXCEPTO un reinicio físico — que está cubierto por Windows automáticamente.

Test 1 — Cerrá TODAS las ventanas de PowerShell

Cerrá la PowerShell admin que estuviste usando. Cerrá cualquier otra que tengas abierta. Desde tu **celular con datos móviles**:

```
https://api.tsicr-api.uk/health
```

[OK] Debe responder

```
{"ok":true,...}
```

Por que importa: los servicios corren bajo LocalSystem, no bajo tu usuario.

Test 2 — Desconectate del Escritorio Remoto

- Cerrá la conexión RDP normalmente (X en la ventana, o *Disconnect*).
- **NO te deslogues** (que sería "Sign out" — eso es Test 3). Solo desconectá.
- Esperá 30 segundos.
- Desde tu celular: `https://api.tsicr-api.uk/health`

Por qué importa: muchos setups mal configurados se caen cuando se desconecta RDP porque dependen de la sesión activa. Como InforBridge y cloudflared corren bajo LocalSystem, esto no les afecta.

Test 3 — Cerrar sesión de Windows (Sign out)

[!] **HACELO SOLO si nadie más está conectado por RDP a la laptop.** Esto NO afecta a Infor — la BD sigue funcionando aunque cierres sesión.

- Volvé a conectarte por RDP.
- **Inicio » tu nombre » Sign out.**
- Esperá 30 segundos.
- Desde tu celular: `https://api.tsicr-api.uk/health`
- Volvé a conectarte por RDP. Vas a tener que loguearte de nuevo.

Por qué importa: confirma definitivamente que los servicios NO dependen de ningún usuario logueado. Es lo más cerca de "simular un reboot" sin reiniciar.

Test 4 — Reiniciar manualmente los servicios

```
Restart-Service InforBridge
Start-Sleep -Seconds 5
Restart-Service cloudflared
Start-Sleep -Seconds 10
Get-Service InforBridge, cloudflared
```

Ambos deben terminar en **Running**. Después:

```
curl.exe https://api.tsicr-api.uk/health
```

Por qué importa: simula "el servicio crasheó y Windows lo reinició solo". Si funciona, tu auto-restart está validado.

E.4 Estado final esperado

Cerrás todas las PowerShell	[OK] Sí
Te desconectás de RDP (sin sign out)	[OK] Sí
Cerrás sesión de Windows	[OK] Sí
Un servicio crashea	[OK] Sí (auto-restart en 5s)
Reiniciás la laptop (cuando puedas)	[OK] Sí (servicios Automatic arrancan al boot)
Se corta la luz	[!] Sí, si la laptop arranca sola (UPS o BIOS)

F. PARTE F — Verificación final desde internet

Tiempo: **10 minutos**. Hacelas todas **desde fuera** de la red de TechShop (datos móviles del celular, casa, otra red). Si cada una pasa, estás listo para PARTE H (integración con TechApp_Back).

[i] Primero definí \$T en PowerShell para no repetir la API key:

```
$T = "PEGA_TU_API_KEY"
$URL = "https://api.tsicr-api.uk"
```

F.1 Health (sin auth)

```
curl.exe "$URL/health"
```

[OK] Esperás:

```
{"ok":true,"db":"TechShop_App",...}
```

F.2 Listar tablas permitidas

```
curl.exe -H "x-api-key: $T" "$URL/api/tables"
```

[OK] Esperás:

```
{"read":["BridgeTest","Productos",...],"write":["BridgeTest",...]}
```

F.3 Leer datos reales (paginado)

```
curl.exe -H "x-api-key: $T" "$URL/api/tabla/BridgeTest?limit=3"
```

[OK] Esperás:

```
{"rows":[{"Id":1,...}, {"Id":2,...}], "count":3, "limit":3, "offset":0}
```

F.4 Leer una fila puntual

```
curl.exe -H "x-api-key: $T" "$URL/api/tabla/BridgeTest/1?pk=Id"
```

[OK] Esperás:

```
{"Id":1,"Descripcion":"fila uno","Numero":1}
```

F.5 SELECT con JOIN parametrizado

```
$body = '{"sql":"SELECT TOP 5 Id, Descripcion FROM BridgeTest WHERE Numero >=
@n","params":{"n":1}}'
curl.exe -X POST -H "x-api-key: $T" -H "Content-Type: application/json" `
-d $body "$URL/api/select"
```

[OK] Esperás:

```
{"rows": [...], "count": N}
```


F.6 Modificar datos (UPDATE)

```
$body = '{"pk":"Id","set":{"Descripcion":"prueba desde internet","Numero":99}}'
curl.exe -X PUT -H "x-api-key: $T" -H "Content-Type: application/json" `
-d $body "$URL/api/tabla/BridgeTest/1"
```

[OK] Esperás:

```
{"ok":true,"affected":1,"tabla":"BridgeTest","id":"1"}
```

Verifica en SSMS que el cambio se persistió:

```
SELECT * FROM TechShop_App.dbo.BridgeTest WHERE Id = 1;
```

La fila 1 debe tener Descripcion = 'prueba desde internet' y Numero = 99.

F.7 Tests de bloqueo (deben devolver 403)

```
# DELETE bloqueado
curl.exe -X DELETE -H "x-api-key: $T" "$URL/api/tabla/BridgeTest/1"
# Esperás: {"error":"DELETE no permitido"}

# POST/INSERT bloqueado
curl.exe -X POST -H "x-api-key: $T" "$URL/api/tabla/BridgeTest"
# Esperás: {"error":"INSERT no permitido"}

# UPDATE en tabla NO whitelisted
curl.exe -X PUT -H "x-api-key: $T" -H "Content-Type: application/json" `
-d '{"pk":"Id","set":{"X":"Y"}}' "$URL/api/tabla/Facturas/1"
# Esperás: {"error":"tabla no permitida para modificación"}

# Sin API key
curl.exe "$URL/api/tabla/BridgeTest"
# Esperás: {"error":"unauthorized"}
```

[X] Los 4 deben fallar con 403 (o 401 el último). Si alguno deja pasar, **parate y revisá server.js** — algo está mal configurado.

G. PARTE G — Laptop como servidor 24/7

Tiempo: **15 minutos**. Adaptado para no requerir reinicio.

[X] Esta fase es **OBLIGATORIA**. La causa #1 de fallas en este tipo de setup es que la laptop entró en suspensión o hibernación. Configuraré esto antes de dar por hecho el sistema.

G.1 Desactivar suspensión e hibernación

Configuración » **Sistema** » **Inicio/apagado y batería** (o Power & battery). Poné los 4 desplegables en **Nunca**:

- Con batería, apagar pantalla después de: **Nunca**.
- Conectado, apagar pantalla después de: **Nunca**.
- Con batería, suspender después de: **Nunca**.
- Conectado, suspender después de: **Nunca**.

Apagar hibernación con PowerShell admin:

```
powercfg /hibernate off
```

Verificalo:

```
powercfg /availablesleepstates
```

Hibernate debe figurar en "The following sleep states are NOT available". Si todavía aparece como disponible, repetí el comando.

G.2 Cerrar la tapa NO suspende

Si la laptop tiene tapa (no es desktop):

- Panel de Control » **Hardware y sonido** » **Opciones de energía**.
- Click en **Elegir el comportamiento del cierre de la tapa**.
- Al cerrar la tapa: **No hacer nada** » con batería Y conectado.
- Guardar cambios.

G.3 Windows Update controlado (no reboot automático)

Sin esto, Windows Update reinicia la laptop sin avisar a las 3 AM y rompe el servicio.

- Configuración » **Windows Update** » **Opciones avanzadas**.
- Horas activas: **Manualmente**, rango 5:00 AM a 11:00 PM. Así Windows solo puede reiniciar entre 11 PM y 5 AM (cuando la decisión es tuya).
- Pausar actualizaciones: **5 semanas** (el máximo).
- Activar: **Avisarme cuando se requiera reinicio**.

[i] Como NO podés reiniciar bajo demanda: una vez al mes entrás a Windows Update, anotás qué patches están pendientes, y los aplicás cuando puedas programar un mantenimiento (finde, vacaciones, etc).

G.4 BIOS auto-power-on tras corte de luz (DIFERIDO)

[!] Este paso requiere reiniciar la laptop para entrar al BIOS. Como no podés reiniciar ahora, anotalo para cuando tengas la próxima ventana de mantenimiento programado. Mientras tanto, mitigá el riesgo con G.5 (UPS).

Cuando puedas:

- Reiniciá la laptop.
- Apenas arranque, apretá la tecla del BIOS (HP=F10, Dell=F2, Lenovo=F1, Asus=F2, Acer=F2/Del).
- Buscá **AC Power Recovery**, **Restore on AC Power Loss** o **After Power Failure** (suele estar en Power Management o Advanced).
- Cambiá a **Power On** (o Last State).
- F10 » Save & Exit.

G.5 UPS — la solución sin reboot

[OK] **Recomendación FUERTE:** comprá una UPS pequeña (\$40-80 USD). Sin BIOS auto-power-on (G.4), la UPS es tu único seguro contra cortes de luz. Modelos: APC Back-UPS BX700U-LM, Forza FX-750LCD-U, CDP B-Smart 750.

Una UPS de 700 VA aguanta una laptop ~30-60 minutos. La mayoría de cortes domésticos/industriales son cortos (< 15 min), así que la laptop nunca se apaga.

G.6 UptimeRobot (opcional pero recomendado)

Te avisa por mail apenas la API deja de responder.

- Cuenta gratis en uptimerobot.com.
- Add New Monitor » HTTP(s).
- Friendly Name: API Infor
- URL: <https://api.tsicr-api.uk/health>
- Monitoring Interval: **5 minutes**.
- Alert Contacts: tu email.
- Save.

G.7 Reglas de oro para la laptop

- [X] NO apagues la laptop nunca.

- [X] NO desenchufes el cargador.
- [X] NO cierres la tapa esperando suspensión (ya configuraste tapa = sigue corriendo).
- [X] NO toques los servicios InforBridge ni cloudflared en services.msc.
- [X] NO instales programas pesados en esta laptop.
- [OK] Sí revisá Windows Update una vez al mes en horario que vos elijas.
- [OK] Sí chequeá el dashboard de UptimeRobot una vez por semana.
- [OK] Sí pegá un cartel en la laptop: "NO APAGAR — SERVIDOR".

H. PARTE H — Integración con TechApp_Back

Esta fase la **armamos juntos** cuando me confirmes que PARTE F pasó (verificación final desde internet). Acá te dejo el panorama de qué se va a crear.

H.1 Variables de entorno

En el `.env` local de TechApp_Back y en Railway:

```
INFOR_API_URL=https://api.tsicr-api.uk
INFOR_API_KEY=el_mismo_token_del_bridge
```

H.2 Estructura de archivos a crear

```
TechApp_Back/src/
config/
inforApi.ts <- cliente axios con x-api-key
modules/
infor/
infor.controller.ts <- handlers HTTP
infor.routes.ts <- montado en /api/infor
infor.service.ts <- métodos tipados (getProductos, etc.)
infor.schemas.ts <- validación Zod de body/params/query
infor.types.ts <- interfaces tipadas por tabla
```

H.3 Endpoints que va a exponer TechApp_Back

Tu frontend nunca llama directo a `api.tsicr-api.uk`. Siempre pasa por TechApp_Back, que ya tiene auth (JWT), rate limiting y logging.

Método	Ruta	Acción
GET	/api/infor/health	Verifica que el bridge responde
GET	/api/infor/tables	Lista whitelist (debug)
GET	/api/infor/tabla/:nombre	Lista filas paginadas
GET	/api/infor/tabla/:nombre/:id	Una fila
POST	/api/infor/select	SELECT libre (con guardas)
PUT	/api/infor/tabla/:nombre/:id	UPDATE de campos

H.4 Cómo se va a usar desde el código

Ejemplos del service que voy a generar:

```
// Lectura paginada
const productos = await inforService.list('Productos', { limit: 50 })
// productos = { rows: [...], count: 50, limit: 50, offset: 0 }

// Una fila
```

```

const p = await inforService.findById('Productos', 123)

// SELECT con JOIN parametrizado
const activos = await inforService.select(
  'SELECT TOP 10 Id, Nombre FROM Productos WHERE Activo = @act',
  { act: 1 }
)

// UPDATE
await inforService.update('Productos', 123, {
  Descripcion: 'nuevo texto',
  Precio: 1500
})

```

H.5 Lo que necesito de vos para arrancar PARTE H

Cuando llegues a este punto, mandame:

- Confirmación de que las verificaciones F.1 a F.7 pasaron.
- **API_KEY** generada (mensaje privado, ahora vamos a meterla en Railway también).
- Lista **final** de tablas en READ y WRITE de tu .env del bridge.
- Para cada tabla que vamos a tipar: las 3-5 columnas más importantes con su tipo (ej: Id INT, Nombre NVARCHAR(100), Activo BIT). Con eso te genero las interfaces TypeScript.

Una vez que tenga eso, te entrego en una sola tanda:

- Los archivos del módulo infor/ creados en TechApp_Back.
- Las nuevas variables agregadas a .env y al validador config/env.ts.
- Tests de integración mínimos con vitest.
- Documentación inline (TSDoc) de cada método.

Apéndice A — Troubleshooting

[D.2] cloudflared no es reconocido como comando

El PATH no se actualizó. Cerrá TODAS las ventanas de PowerShell y abrí una nueva. Si igual no funciona, reiniciá la laptop (única excepción a no-reboot — programalo).

[D.3] cloudflared tunnel login no abre el navegador

Bloqueador de popups o antivirus. Copiá la URL larga que aparece en consola y pegala manualmente en el navegador.

[D.4] error parsing YAML in config file

Indentación con tabs en lugar de espacios. YAML rechaza tabs. Reescribí los espacios a mano (2 por nivel).

[D.4] tunnel ID does not match

El UUID en `tunnel:` no coincide con el del archivo `.json`. Verificá ambos contra `cloudflared tunnel list`.

[D.5] failed to add route: code 1003 already exists

El subdominio `api` ya tiene otro registro. Andá al dashboard CF » DNS, borrá el registro existente, volvé a correr el comando.

[D.6] failed to dial to edge: dial udp ... i/o timeout

Firewall bloqueando UDP/QUIC. Probá: `cloudflared tunnel run --protocol http2 infor-bridge`

[D.6] Error 1033 en el celular

El túnel no está activo. Volvé a la Ventana 2 (cloudflared) y verificá que diga 'Connection registered'.

[D.6] Error 502 Bad Gateway

El bridge (Ventana 1) no está corriendo o crasheó. Volvé a la Ventana 1.

[D.6] Error 530 Origin Unreachable

El `config.yml` apunta a un puerto/host equivocado. Verificá `service:` `http://localhost:8787`.

[D.6] DNS_PROBE_FINISHED_NXDOMAIN

DNS aún no propagó. Esperá 2-5 minutos más, refrescá.

[D.6] Error 1016 Origin DNS error

El proxy en Cloudflare está apagado (nube gris). Volvé a D.5.2 y poné la nube en NARANJA.

[E.1] El servicio InforBridge dice Stopped

Logs en `C:\infor-bridge\service-err.log`. Error más común: NSSM no encontró `node.exe`. Verificá `Get-Command node` y recreá con `nssm.exe edit`.

[E.2] cloudflared como servicio dice 0 conexiones

El servicio busca el config en `C:\Windows\System32\config\systemprofile\.cloudflared\`, no en tu user profile. Copiá los archivos ahí (ver E.2.4).

[F] Login failed for user 'sa'

Contraseña en `.env` distinta de la real. Probá la contraseña directo en SSMS primero. Confirmá auth mixta. Reiniciá InforBridge tras cambiar `.env`.

[F] tabla no permitida (caso correcto)

Mayúsculas/minúsculas distintas. Productos != productos. Editá `.env`, agregá la tabla con grafía exacta de SQL Server, `Restart-Service InforBridge`.

[F] UPDATE devuelve affected: 0

La fila con ese Id no existe, o el campo pk está mal. Por defecto es Id; si tu tabla usa otro nombre, pasalo en el body: `{"pk": "CodigoProducto", "set": {...}}`

[General] Quiero ver qué queries se ejecutan

Logs del servicio: `C:\infor-bridge\service-out.log`. En vivo: `Get-Content C:\infor-bridge\service-out.log -Tail 50 -Wait`

[General] Cómo rotar el API_KEY

Editá `C:\infor-bridge\.env`, generá nuevo token, guardar. `Restart-Service InforBridge`. Actualizá `INFOR_API_KEY` en Railway y en `.env` local del backend.

Apéndice B — Referencia rápida

B.1 Endpoints del bridge

Método	Ruta	Auth	Acción
GET	/health	No	Estado del servicio
GET	/api/tables	x-api-key	Lista whitelist
GET	/api/tabla/:nombre	x-api-key	Lee filas paginadas
GET	/api/tabla/:nombre/:id	x-api-key	Lee una fila por PK
POST	/api/select	x-api-key	SELECT libre
PUT	/api/tabla/:nombre/:id	x-api-key	Actualiza una fila
POST	/api/tabla/*	—	403 (INSERT bloqueado)
DELETE	/api/tabla/*	—	403 (DELETE bloqueado)

B.2 Comandos del día a día

```
# Estado de los servicios
Get-Service InforBridge, cloudflared

# Reiniciar bridge tras cambiar .env
Restart-Service InforBridge

# Reiniciar túnel
Restart-Service cloudflared

# Logs del bridge en vivo
Get-Content C:\infor-bridge\service-out.log -Tail 50 -Wait

# Errores recientes
Get-Content C:\infor-bridge\service-err.log -Tail 50

# Estado del túnel + conexiones
cloudflared tunnel list

# Health local
curl.exe http://localhost:8787/health

# Health desde internet
curl.exe https://api.tsicr-api.uk/health
```

B.3 Ejemplos curl listos

Listar 50 filas

```
$T = "PEGA_TU_API_KEY"
$URL = "https://api.tsicr-api.uk"
curl.exe -H "x-api-key: $T" "$URL/api/tabla/Productos?limit=50&orderBy=Id"
```

Una fila puntual

```
curl.exe -H "x-api-key: $T" "$URL/api/tabla/Productos/123?pk=Id"
```

SELECT con WHERE parametrizado

```
$body = '{"sql":"SELECT TOP 10 p.Id, p.Nombre FROM Productos p WHERE p.Activo = @act","params":{"act":1}}'
curl.exe -X POST -H "x-api-key: $T" -H "Content-Type: application/json" `
-d $body "$URL/api/select"
```

Actualizar una fila

```
$body = '{"pk":"Id","set":{"Descripcion":"nuevo texto","Precio":1500}}'
curl.exe -X PUT -H "x-api-key: $T" -H "Content-Type: application/json" `
-d $body "$URL/api/tabla/Productos/123"
```

Apéndice C — Checklist final

Imprimí o ticá los puntos a medida que avances.

PARTE D — Cloudflare Tunnel

- ☐ cloudflared --version responde
- ☐ cert.pem creado en .cloudflared
- ☐ Túnel infor-bridge creado, UUID anotado
- ☐ config.yml validado con 'ingress validate' = OK
- ☐ DNS route apuntado a api.tsicr-api.uk
- ☐ CNAME en Cloudflare con nube NARANJA (Proxied)
- ☐ Bridge corriendo (Ventana 1)
- ☐ Cloudflared corriendo con 4 conexiones (Ventana 2)
- ☐ Health responde desde celular con DATOS MÓVILES
- ☐ GET /api/tabla/X con API key responde con filas

PARTE E — Servicios Windows

- ☐ NSSM extraído en C:\nssm\
- ☐ Servicio InforBridge instalado
- ☐ Auto-restart configurado (sc.exe failure)
- ☐ Get-Service InforBridge = Running
- ☐ curl localhost:8787/health = ok
- ☐ cloudflared service install ejecutado
- ☐ Get-Service cloudflared = Running
- ☐ cloudflared tunnel list muestra conexiones > 0
- ☐ Test 1: cerré PowerShell y la API responde desde celular
- ☐ Test 2: desconecté RDP y la API sigue respondiendo
- ☐ Test 4: Restart-Service y la API vuelve sola

PARTE F — Verificación final desde internet

- ☐ F.1 GET /health = ok:true
- ☐ F.2 GET /api/tables = listas read/write
- ☐ F.3 GET /api/tabla/X?limit=3 = filas reales
- ☐ F.4 GET /api/tabla/X/:id = una fila
- ☐ F.5 POST /api/select = JOIN parametrizado funciona
- ☐ F.6 PUT /api/tabla/X/:id = UPDATE persiste en SSMS

- [] F.7 DELETE / POST / tabla no whitelisted = 403

PARTE G — Laptop 24/7

- [] Suspensión configurada en Nunca (4 desplegables)
- [] Hibernación apagada (powercfg /hibernate off)
- [] Cerrar tapa = No hacer nada
- [] Windows Update con horas activas controladas
- [] Pausa de actualizaciones a 5 semanas (renovar mensual)
- [] (Opcional) UPS comprada e instalada
- [] (Diferido) BIOS auto-power-on cuando puedas reiniciar
- [] (Opcional) UptimeRobot monitoreando /health
- [] Cartel en la laptop: 'NO APAGAR — SERVIDOR'

PARTE H — Integración TechApp_Back

- [] Lista final de tablas READ/WRITE consensuada
- [] Tipos de columnas anotados para cada tabla principal
- [] INFOR_API_URL e INFOR_API_KEY en .env local
- [] Las mismas vars subidas a Railway
- [] Módulo /api/infor/* creado y deployado
- [] Endpoint /api/infor/health responde 200
- [] Frontend consumiendo datos reales de Infor

Cualquier paso que falle, parate y consultame antes de avanzar. Avanzar con un paso roto multiplica los problemas más adelante.